

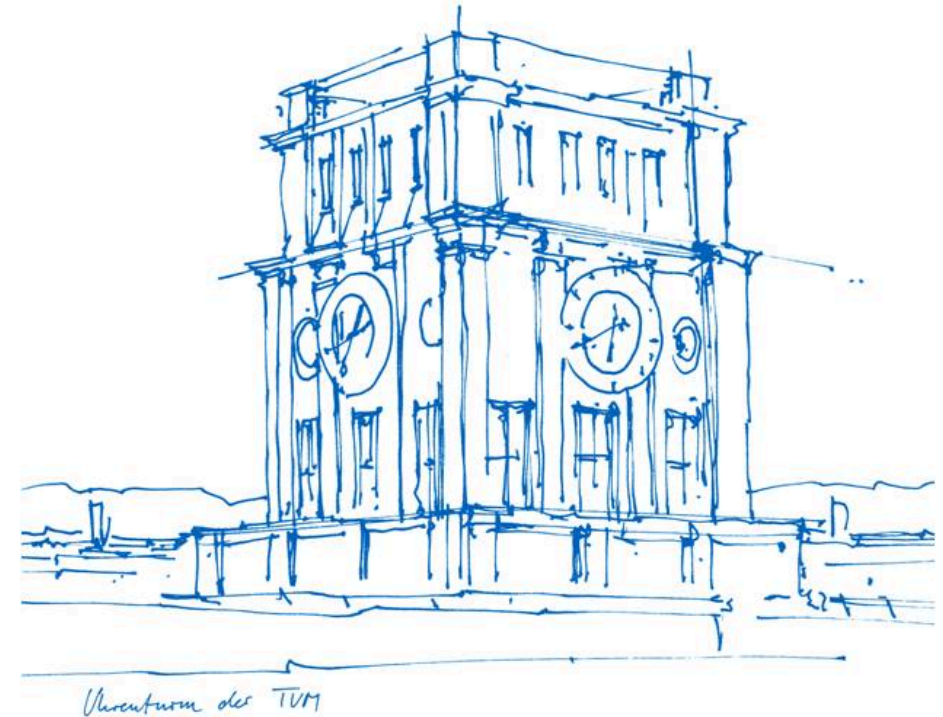
Agentic AI in Modern Business

A GUI agent for AndroidWorld, and where it breaks

Group project

Jonas Schmidt · Vincent Fischer · Liam Meinhart
Bela Gehlen · Gloria Dell'Etere

Technische Universität München · Campus Heilbronn
Agentic AI in Modern Business · Group Project · 09.10.2026



AGENDA

Five chapters, one story

- 1 The assignment**
What we were asked to build, and the three tasks we chose to run it on.
- 2 How we built it**
One agent, six versions. The decisions behind it, from the model to the guardrails.
- 3 How we measured**
When a run stops, what counts as a pass, and the one row we wrote per run.
- 4 What happened**
The V1 numbers, all 81 runs in one matrix, one failure step by step, and the diagnosis.
- 5 What we changed and learned**
One change at a time, the before-and-after numbers, where a fix belongs, and what is left open.

1

The assignment

What we were asked to build, and the three tasks we chose to run it on.

Build an agent that operates a phone, measure it, improve it

THE TASK

- A GUI agent for AndroidWorld
- One class, one `step()`
- The agent only sees pixels and buttons

THE GOAL

- Not a demo: a measured result
- 3 tasks × 3 runs × 6 versions
- Failures explained, every version measured before and after

THE GUIDELINES

- Any model, but name it
- Guardrails in code, not in the prompt
- One task per difficulty row

The three tasks: a contact, a note, a note sent by SMS

Difficulty	Task	What the agent is told	Budget
Warm-up	ContactsAddContact	Create a new contact for {name}. Their number is {number}.	12
Real work	MarkorCreateNote	Create a new note in Markor named {file_name} with the following text: {text}	16
Multi-app & memory	MarkorCreateNoteAndSms	Create a note named {file_name} with {text}. Share its entire content with {number} via SMS.	18

2

How we built it

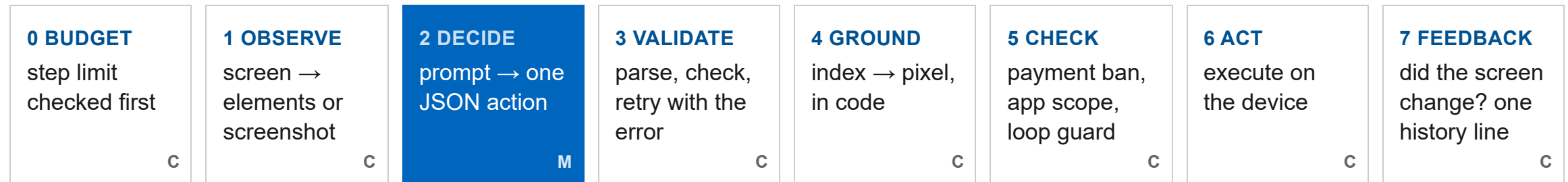
One agent, six versions. The decisions behind it, from the model to the guardrails.

Which model we use and how it is configured

Model	qwen3-vl:4b-instruct, 4.4B parameters
Where it runs	laptop, Ollama server at localhost:11434
Quantisation	Q4_K_M, 4 bits per weight
Memory for weights	$4.4\text{B} \times 0.5 \text{ byte} + \text{projector} \approx \mathbf{3.3 \text{ GB}}$
Context window	num_ctx 8,192 tokens (Ollama default 4,096 is too small for tree + image)
Sampling	temperature 0 , seed 42 , top-p left at default
Output	max 400 tokens, JSON-schema constrained

- Free after download, slow, small only
- Temperature 0 + fixed seed, so a result can be repeated
- The variant that answers at once, not the one that thinks first
- Every run records name, size, quantisation and digest

Agent design: the eight stages of one step()



C = code we wrote · M = the model

Harness part	What it decides	Where in step()
Loop control	when to call the model again, when to stop	stage 0, stop rules
Context assembly	what goes into the window, inside the budget	stage 1, prompts.py, memory.py
Tool dispatch	turn the JSON into a real tap on the device	stages 4 and 6
Error recovery	parse failed, no effect: retry, repair or give up	stages 3 and 7
Guardrails	what is forbidden, what gets logged	stage 5, guardrails.py

One step of the ReAct loop: reason → act → observe

```

REASON  "thought": "I need to create a new contact for
          Hugo Pereira. The 'Create contact' button is
          visible and accessible."
ACT      "action_type": "click", "index": 1
OBSERVE  screen changed → new tree, 19 elements
          history: 5. click [1] "Create contact" -> executed
    
```

Step 5 of our ContactsAddContact run. One model call, one action, one observation, then the next call with a longer history.

Phase	Term	In our agent
Reason	no tool	the model's thought, logged
Act	Execution tools	click, long_press, input_text, scroll, open_app, back, home, enter
Act	Collaboration, user communication	none; infeasible is the only way to give up
Observe	Perception tools	read the screen: tree or screenshot
Act + Observe	Plugin ("Computer Use")	AndroidWorld supplies both spaces
before the loop	Tool discovery, MCP	not needed: ten fixed actions

Pixels see everything but point badly. The tree is precise but incomplete.

Screenshot V1

- Sees every pixel, also WebViews and drawings
- ~1,900 prompt tokens per step
- The model has to find the row itself

Accessibility tree V2 V3 V4 V5 V6

```
[1] ImageButton "Create contact"  
[2] Button "Don't allow"  
[7] EditText "First name"  
[10] EditText "Phone"
```

- ~1,000 to 1,070 prompt tokens per step, exact bounds
- Whatever has no description is simply not there
- V4 to V6 keep the tree and change what is around it (chapter 5)

How the agent points: coordinates vs. index

By coordinate V1

```
{"action_type": "click", "x": 540, "y": 735}
```

The model produces the numbers. A 20-pixel error is a tap on the gap between two rows, and nothing reports an error.

By index V2 V3 V4 V5 V6

```
{"action_type": "click", "index": 3}
```

Our code looks element 3 up in the tree. It either finds it or it doesn't: a failure we can see and log.

Old screens are never resent: one line per step, status last

```
system  role · action space · rules      (stable prefix, cached)
user    GOAL: Create a new contact for Hugo Pereira ...
        HISTORY:
        1. navigate_home -> executed
        2. click [6] "Search" -> BLOCKED: home screen
        3. open_app contacts -> executed
        CURRENT SCREEN: [0] ... [13]      (this step only)
        STATUS: step 4 of 12 · app: contacts
        note: number +13920741751        (V3 only, most volatile, last)
```

- A tool-call loop, but compressed
each step gets a fresh prompt: the old screen is dropped, one text line per step stays. 20 kept screenshots would cost 20,000+ tokens
- Stable prefix first, volatile last, so the KV cache hits
the model reuses what it computed for an unchanged beginning: fixed rules first, the ever-changing status line last
- History and status are written by code, not by the model
"BLOCKED" or "changed nothing" is recorded by our code, so it is a fact. V3 adds a note line, V4 a tracker block, V6 a done hint: each one a line written by code

The action space: ten actions, one schema

click · long_press · input_text · scroll
open_app · navigate_back · navigate_home
keyboard_enter · wait · status

```
"action_type": {"enum": [the ten above]}  
"index":      {"type": "integer"}  
"app_name":   {"enum": ["contacts", "markor",  
                        "simple sms messenger", ...]}  
"goal_status": {"enum": ["complete", "infeasible"]}
```

- Schema handed to Ollama for constrained decoding
- open_app only by exact name from the allow-list
- Tolerant parsing, strict validation, one retry with the error

Guardrails: in the code, before the action, no off switch

- Step limit
- Payment ban, layer 1: app scope
- Payment ban, layer 2: deny rules
- Loop guard **V3**

The first three are identical in all six versions and never switched off.

A guardrail firing in our run **V2**

2. click [6] "Search"
-> **BLOCKED: do not tap or type on the home screen.**
Open the app you need with open_app and its exact name.
3. open_app contacts -> executed

Step 2: the model wanted to tap the home-screen search. The code refused, and told the model why. Step 3 did it right.

3

How we measured

When a run stops, what counts as a pass, and the one row we wrote per run.

HOW WE MEASURED · WHEN A RUN STOPS, AND WHAT COUNTS

A run ends for one of five reasons. Only one of them can be a PASS.

Stop reason	Who decided	Can be PASS?
model_done	model says status: complete	yes, if the verifier agrees
model_infeasible	model gives up	no
step_limit	code: budget reached	no, even if the screen is right
loop_abort	code: same action, same screen, 3× blocked (V3)	no
error	emulator or model server broke	ERROR, excluded from rates

PASS means both:

- the agent itself reported *complete*
- AndroidWorld's checker returns 1.0 on the device state

Running out of steps is never counted as done, and a correct screen without the report is still a FAIL: the agent has to know it is done.

HOW WE MEASURED · ONE ROW PER RUN

Written while the run happens, in the lecture's format

task	ContactsAddContact
run	setup run (not scored)
model	qwen3-v1:4b-instruct (Q4_K_M, local, Ollama)
observation	accessibility tree
grounding	by index
steps	11 (budget 12)
verifier	FAIL
failure_class	lost_value
recording	runs_setup_windows/v2_index/ContactsAddContact/run1_20261008-103339/
note	typed "Hugo", never "Pereira"; reported complete anyway

Written into runs/log.csv the moment a run ends, with a suggested failure class. A human confirms it from the recording (failure_reviewed = yes).

Failure class: one of five words, fixed list, one per run
grounding · too_early · lost_value · wrong_app · false_done

4

What happened

The V1 numbers, one failure step by step, and the diagnosis.

WHAT HAPPENED · RESULTS, V1 BASELINE

V1 results: 5 of 9 pass, every failure is in the two harder tasks V1

Task	Run	Steps (budget)	Verifier	Failure class	Note
ContactsAddContact	1/3	9 (12)	PASS		
ContactsAddContact	2/3	8 (12)	PASS		
ContactsAddContact	3/3	9 (12)	PASS		
MarkorCreateNote	1/3	10 (16)	PASS		
MarkorCreateNote	2/3	11 (16)	PASS		
MarkorCreateNote	3/3	9 (16)	FAIL	false_done (?)	reported complete without ever tapping Save
MarkorCreateNoteAndSms	1/3	17 (18)	FAIL	grounding (?)	note saved (credit 0.5); 1 tap on no element in the SMS app
MarkorCreateNoteAndSms	2/3	18 (18)	FAIL	grounding (?)	note saved (credit 0.5); 5 taps on the same spot in the SMS app, no effect
MarkorCreateNoteAndSms	3/3	18 (18)	FAIL	grounding (?)	number never typed; 4 taps on the same spot; out of steps

Windows laptop, 08.10.2026, 0 ERROR runs. (?) = suggested by the runner, to be confirmed against the recording.

WHAT HAPPENED · ALL RUNS, ONE MATRIX

All 81 scored runs: nine task instances, nine version-machine columns

task instance (same seed in every column)	W V1	W V2	W V3	M V1	M V2	M V3	M V4	M V5	M V6
Contacts · Hugo Pereira	P	F lv	F gr	P	P	F lv	F gr	F gr	F gr
Contacts · Mariam Chen	P	F lv	F lv	F gr	F gr	F te	F gr	P	F gr
Contacts · Isla Martin	P	F lv	F lv	P	F lv	F lv	F gr	P	F gr
Markor · wise_yacht.md	P	P	F gr	P	F gr	F gr	P	P	F gr
Markor · brave_nest.txt	P	P	F gr	P	F gr	F gr	F gr	F gr	F gr
Markor · quick_eagle.md	F fd	P	F gr	F fd	P	F gr	F gr	F gr	P
Note+SMS · fair_ant.txt	F gr	F gr	F gr	F gr	F lv	F gr	F gr	F gr	F gr
Note+SMS · fierce_violin.txt	F gr	F lv	F gr	P	F gr	F gr	F gr	F gr	F gr
Note+SMS · lively_goat.md	F gr	F fd	F gr	F gr	F fd	F gr	F gr	F gr	F fd
PASS	5	3	0	5	2	0	1	3	1

W = Windows laptop, M = MacBook Air. V4 to V6 are three later single-change iterations on the Mac, explained in chapter 5. gr grounding · lv lost_value · fd false_done · te too_early. Classes are the log's; the runs discussed in this talk are reviewed. Read down a column for a version's pattern, across a row for the noise between machines and versions.

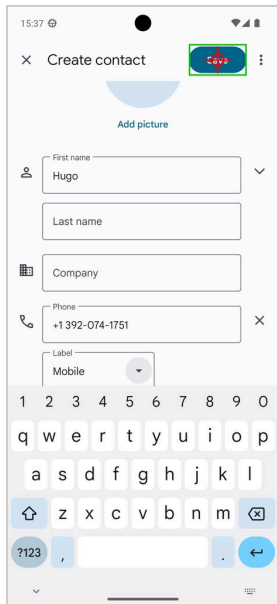
The three most common failures of each version, in plain words

		1	2	3
V1	V1	Coordinate taps that change nothing, mostly in the SMS app (4 to 6 per run)	Number never typed, budget runs out in the SMS app	"complete" without ever tapping Save (Markor run 3)
V2	V2	Surname skipped: first name, phone, Save, "complete" (W 3 of 3, M 2 of 3 Contacts runs)	File name typed with its extension into the name field while the extension field says .md (.txt tasks)	Correct screen, "done" never said; Save tapped 3 to 5× without visible effect
V3	V3	Save loop: "no effect" hint + loop guard fire on Markor's silent autosave, then detours (Share menu, wrong SMS app, Reload)	Check step accepts the missing surname every time it is reached	Coordinate tap opens "Add photo", typing lands in the camera app; menu detours eat the budget
V4	V4	Home-screen loop: "back" 11× with the identical thought, open_app forgotten (3 of 3 Contacts)	Extension field tapped 10 to 15× instead of typed into	Correct state, "done" never said (SMS run 3)
V5	V5	Extension field tapped 6 to 14× (Note+SMS runs 1, 2)	Correct state, "done" never said; Reload menu opened instead (Markor 3, SMS 3)	Photo-area detour costs the last step before the number (Contacts 1)
V6	V6	Extension field and "create file" button tapped 6 to 14× (4 runs)	Done hint read, button pressed again anyway; on Markor Save the hint never fires	Home-screen back loop and waits (Contacts 1, 2); correct state at step 12 of 12 (Contacts 3)

Two patterns survive every version from V2 on: the extension field and the missing "done". One pattern (the lost surname) disappears only with the 8B model.

WHAT HAPPENED · ONE FAILURE, STEP BY STEP

It typed "Hugo", never "Pereira", and said it was done V2



the 11 steps, one frame each

2 click [6] "Search" on the home screen → **blocked** by the harness

3 open_app contacts → executed

7 input_text "Hugo" into First name

9 input_text "+13920741751" into Phone

10 click [2] "Save"

11 status complete → verifier: **FAIL**

Model's own thought at step 11: "The contact has been successfully created."

Class: **lost_value**: a value from the goal never reached the device; the silent "done" is the symptom. Not a one-off: the surname is missing in 5 of 6 tree-based 4B runs (V2, V3); V1 typed both names every time, and so did the 8B model (V5, V6: 0 of 6 lost).

WHAT HAPPENED · WAS IT RELIABLE?

Reliability check: strong on constraints, weak on verification. V3 added it, and it misfired.

Reliability lever	What it asks	V1 / V2	V3
Goal clarity	What counts as success?	✓ the task text, plus AndroidWorld's checker on the device state	✓ same
Observable state	What can the agent see?	✓ the current screen every step; history lines show what changed	✓ same
Feedback	Does it know what happened?	weak : "executed"	"your action changed nothing": wrong on Save , Markor autosaves without a visible change
Verification	Did the result work?	none before done : the missing surname goes through	check step with the same 4B model: accepted the missing surname every time
Recovery	Retry, revise, roll back or stop?	invalid JSON: retry with the error, 2×. Timeout 300 s: ERROR	+ loop guard: same action, same screen, 3× blocked. Fired on Save , ended 3 runs
Constraints	Which rules must the system enforce?	✓ in code, not in the prompt: step limit, payment ban, app scope	✓ same, held in all 81 runs on two machines

Every V3 mechanism assumes that a successful action changes the screen. Where that assumption fails, the safety net turns a pass into a fail.

WHAT HAPPENED · WAS IT RELIABLE? V4 TO V6 (MACBOOK)

Reliability check, the later iterations: the checks were right, the conditions they waited for never came

Reliability lever	V4 = V2 + goal tracker V4	V5 = V2 + 8B model V5	V6 = V5 + two code rules V6
Goal clarity	✓ same, plus the goal's values listed by code every step	✓ same as V2	✓ same as V2
Observable state	✓ screen + "NOT typed yet: Martin" block	✓ same as V2	✓ same as V2
Feedback	weak : "executed"; nothing says that "back" changed nothing	weak : "executed"	done hint after Save / Send once all values are typed: fired 1× in 9 runs , gated on a visible change
Verification	code veto on "complete" while a value is missing: never reached , the agent never said complete	none before done ; but the 8B model reads the form right: surname 3 of 3	none before done
Recovery	none : 11× back, 10 to 15 taps on one field, no loop guard	none : 6 to 14 taps on the extension field	clear a prefilled field before typing: fired 0× , the loop is taps, not typing
Constraints	✓ held	✓ held	✓ held

V3's lesson, repeated: a mechanism is only as good as the condition it fires on. V4's veto waited for a "complete" that never came; V6's hint waited for a screen change that Markor never shows.

5

What we changed and learned

One change, the before-and-after numbers, and where a fix belongs.

WHAT WE CHANGED AND LEARNED · IN ONE BREATH

Six sentences that 81 runs on two machines support

- Moving grounding into code made every step 3× faster and removed the taps into nothing
V1 → V2, both machines
- With the tree, the 4B model lost the surname in 5 of 6 Contacts runs; the 8B model in 0 of 6
V2 + V3 vs. V5 + V6, MacBook
- Every harness mechanism we added cost steps and none raised the success rate
V3 +1.7, V4 +1.9 steps per run; 2 / 9 → 0 / 9 and 1 / 9
- A correct screen without "done" happened in every version from V2 on
the agent has to know it is finished; the checker does not help it
- Not one task instance passed in all six versions; the .txt notes never passed after V1
same seed, same values: the extension field decides
- The payment ban never had to fire; the home-screen rule did, in V2, V3 and V6
81 runs, 0 payment attempts, 6 blocked home-screen taps on the Mac

WHAT WE CHANGED AND LEARNED · V1 → V6

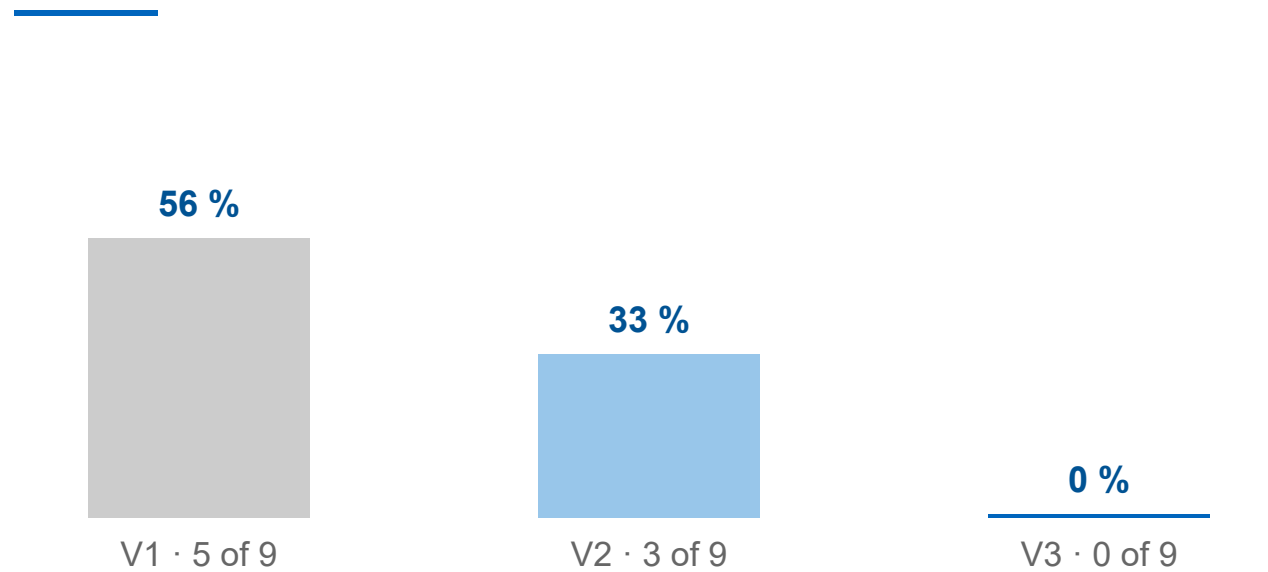
What changed between the versions: one change for V2, five for V3, then three single-change tests

	V1 baseline V1	V2 index V2	V3 full harness V3
Observation	screenshot	accessibility tree	tree
Grounding	coordinates from the model	index, resolved in code	index
Harness additions	none	none	stable-screen wait · "no effect" hint · loop guard · status bar with notes · check before done
Model, temperature, seed	qwen3-vl:4b-instruct , Q4_K_M, local, T = 0, fixed seed	identical	identical
Tasks, task seeds, prompt structure	3 tasks, same 9 task instances	identical	identical
Guardrails	step limit, payment ban, app scope	identical	identical

V1 → V2 is one change, so its effect can be attributed. V2 → V3 stacks five: a package test. MacBook only, one change each on V2: V4 + goal tracker in code · V5 + 8B model · V6 = V5 + clear field, done hint (26b).

WHAT WE CHANGED AND LEARNED · BEFORE / AFTER

V2 is 3× faster and cheaper, and less successful. V3 is worse again.



Success rate (verifier PASS), same 9 task instances per version, 0 ERROR runs. V4 to V6 ran on the MacBook only: slide 26b.

	V1	V2	V3
model time per step	90 s	28 s	27 s
prompt tokens per call	1,905	1,069	1,120
time per run	19 min	7 min	8 min
invalid model replies	0	0	0

WHAT WE CHANGED AND LEARNED · WHERE THE FIX BELONGS

Where the fix belongs: code fixed pointing and app choice, not judgement

Failure	What we tried	Placed in	Did it work?	Where it really belongs
grounding	index instead of coordinates V2	Programs	yes : no more taps into nothing	Programs, as done
wrong_app	open_app by exact name only	Programs	yes : held in all 81 runs on two machines	Programs, as done
lost_value	status bar with notes V3	Programs	no : the surname was never read, not forgotten	Parameters: the 8B model fixed it (V5 V5); a code tracker did not (V4)
false_done	check step before done V3	Programs	no : the checker was the same 4B model	Parameters: a stronger model for the check. Or Programs that read state the UI does not show
Save loop	"no effect" hint + loop guard V3	Programs	backfired : Markor autosaves, no visible change	Instructions: one sentence in a Markor skill file, "Save shows no confirmation"

The lecture's four places for a fix: Knowledge · Instructions · Programs · Parameters. "A rule that must never be bypassed → Programs." A judgement you can state in one sentence → Instructions.

WHAT WE CHANGED AND LEARNED · SAME CODE, SECOND MACHINE

Run again on a MacBook: same ranking, same failure patterns, different single runs

	Windows laptop	MacBook Air M4
V1 v1	5 / 9	5 / 9
V2 v2	3 / 9	2 / 9
V3 v3	0 / 9	0 / 9
V4 · V5 · V6	not run	1 / 9 · 3 / 9 · 1 / 9
ERROR runs	0	11, re-run
V1 model time per step	90 s	23 s
Ollama / model digest	0.40.0 / ef33995b	0.35.1 / ee4b975b

- Same script, same seeds, temperature 0
- Same patterns: the surname goes missing with the tree, V3 loops on Save
- Single runs differ: the model file is not the same build

"If you change quantisation, or switch model, re-run your evaluation. It is a different model." A different digest is a different model.

WHAT WE CHANGED AND LEARNED · ITERATIONS AFTER V3 (MACBOOK)

Three more iterations on the MacBook: a bigger model fixed the lost surname, two code fixes never fired

	V2 V2	V4 = V2 + goal tracker in code V4	V5 = V2 + 8B model V5	V6 = V5 + clear field, "say done" hint V6
success	2 / 9	1 / 9	3 / 9	1 / 9
Contacts	1 / 3	0 / 3	2 / 3	0 / 3
runs with a lost surname	2	0	0	0
model time per step	6.3 s	9.3 s	7.3 s	14.6 s, laptop under load

- V4: the code veto never fired; the 4B model looped instead (12× back, 10 to 15 taps on one field). Every prompt line is an intervention
- V5: +16 % time per step, surname never lost again; the extension-field loop and the missing "done" stay
- V6: the field loop is taps, not typing; the done hint waited for a visible change, Markor's Save gives none (V3's assumption, repeated)

Same model, same seeds, temperature 0: Contacts went 2 / 3 → 0 / 3 between V5 and V6. Three runs find patterns, not rankings.

V4: the code veto never fired. The prompt block did.

V2, Contacts run 1 (Hugo Pereira)

```
1. navigate_home -> executed
2. click [6] "Search" -> BLOCKED: do not tap or type on the
   home screen. Open the app you need with open_app
3. open_app contacts -> executed
... 11. status complete -> PASS
```

V4, same seed, same model

```
1. navigate_home -> executed
2. navigate_back -> executed
3. navigate_back -> executed
...
12. navigate_back -> executed      => FAIL, step_limit
```

The model's own thought, steps 4 to 12, identical every time

"I need to open the Contacts app to create a new contact. Since it's not visible on this screen, I should navigate back to find it."

- It looks for the app on the screen and forgets open_app
- V2 made the same mistake, but tapped: the guardrail answered and the model obeyed. V4 pressed "back": no rule fires, nothing answers
- The tracker block is never mentioned in any thought. The only prompt difference to V2 is that block
- All 3 Contacts runs: 0 / 3, 36 steps, no field ever touched

The 8B model types the surname in 8 steps. The hint it ignores.

V5, Contacts run 3 (Isla Martin), **PASS** in 8 steps

```
3. click [0] "Create contact"
4. input_text "Isla" at x=491 y=387
5. input_text "Martin" into [8] "Last name"
6. input_text "+15256606183" into [10] "Phone"
7. click [2] "Save"
8. status complete -> PASS
```

V2 on the same seed: "Isla", then straight to "Phone", Save, "complete" → FAIL. Same screen, same prompt, 4B vs. 8B.

V6, Note + SMS run 3, the done hint fires once

```
13. input_text "Ignorance is bliss." into [7] "Type a message"
14. click [8] "SMS" -> executed. All values from the goal are
    typed and you pressed a save/send button: if the goal is
    fully done, reply status "complete" now
15. click [8] "SMS" -> executed. (same hint again)
16. navigate_home -> executed
17. status complete -> FAIL, checker 0.5
```

- The hint is read and still the button is pressed again
- On Markor "Save" it never appears: Save changes nothing on screen, and we gated the hint on a change
- Checker 0.5: the SMS shows as sent after the first tap, the second tap hits an empty field. Which half the checker rejected is not visible on screen

WHAT WE CHANGED AND LEARNED · RUN IT YOURSELF

One command per version. The config is the whole agent.

```
python run_eval.py --config configs/v2_index.json
```

- Same task seeds for all six versions (AndroidWorld's own seed formula); one config file per version
- Temperature 0, fixed model seed; model name, size, quantisation and digest recorded per run
"if you change quantisation, or switch model, re-run your evaluation. It is a different model."
- AndroidWorld pinned to one commit; setup scripts for macOS and Windows; 226 unit tests without an emulator
- Every run leaves trajectory.json1, screenshots with the tap marked, recording.mp4, one log row

Five things the runs told us that a demo would not have

- **Where grounding happens matters more than prompt wording**
moving it into code made every step 3× faster and removed the taps into nothing. It also exposed the next weakness: lost values
- **Every safety mechanism encodes an assumption about the app**
V3's hint, loop guard and check step all assume a visible change. Markor autosaves. It became 0 of 9 on both machines
- **A verifier as weak as the actor adds cost, not safety**
the check step was the same 4B model; it accepted the missing surname every time
- **A bigger model closed the knowledge gap, not the finishing gap**
the 8B model never lost the surname (V5, V6); the extension-field loop and the missing "done" stayed
- **Three runs per task find patterns, not rankings**
one run moves a task by 33 points; the trajectories carry the findings, not the rates

What these numbers cannot tell you

Limit	What it means	Evidence
3 runs per task	Rates cannot rank versions; one run moves a task by 33 points	V5 → V6 Contacts 2 / 3 → 0 / 3 with no code change on that path
Two machines, two model files	Same model name, different Ollama build and digest: formally a different model. Single runs are not comparable across machines	Windows ef33995b vs. Mac ee4b975b; Mac V2 2 / 9 vs. Windows 3 / 9
Emulator noise on the Mac	11 crashes re-run, emulator RAM changed 2 → 4 GB mid-measurement, V6 timed under laptop load	runs/log_replaced.csv; V6 14.6 s per step vs. V5 7.3 s, same model
Failure classes are suggestions	Code proposes one of five classes from the trajectory; the six runs discussed in detail were confirmed on the step screenshots, the rest are unreviewed	log.md marks unreviewed rows with (?)
Strict PASS rule	A correct device state without "complete" is a FAIL; the checker also gives 0.5 for a note without the SMS, which we count as FAIL	At least one such run in every version from V2 on
Small local models, three tasks	4B and 8B at 4-bit; no cloud-model baseline, tree + screenshot together never tested, 3 of 116 AndroidWorld tasks	configs/, slide 7

What we would do next, in this order

- **Verify from the device, not from the screen**

read the contacts database and Markor's file list over adb before "done": catches lost_value and false_done with any model (Programs)

- **Observation "both": tree for pointing, screenshot for layout**

the surname was lost only with the tree; the config already has the option, it was never run

- **App knowledge as Instructions, not as loop guards**

one Markor skill file: "Save shows no confirmation; name and extension are separate fields". The mechanism exists (use_skills), off to keep V1 → V2 clean

- **Split actor and verifier**

4B acts, 8B checks: V5 showed the 8B model reads the form correctly, and the check costs one call per run

- **Fix the done hint, then rank properly**

fire on the commit tap, not on a visible change; 10 runs per task on one machine and one digest

TAKEAWAY

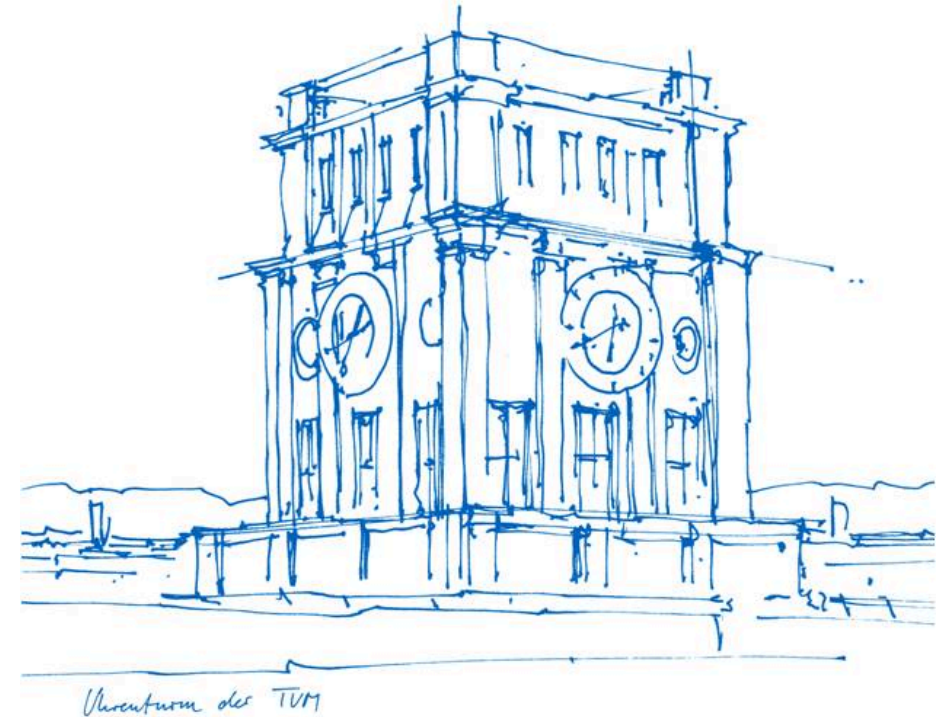
**The model decides. The
harness makes it safe,
measurable and honest.**

Thank you

Questions?

Jonas Schmidt · Vincent Fischer · Liam Meinhart
Bela Gehlen · Gloria Dell'Etere

Technische Universität München · Campus Heilbronn
Agentic AI in Modern Business · Group Project · 09.10.2026



Backup

V2 log rows · V3 · Skills, user memory, RAG · Reward design · Why we did not train · Technical: goal tracker · V6 rules · failure classifier · payment ban · config knobs · all 54 Mac runs · same seed, six versions · where the 81 runs come from · 14 charts re-reading all 81 runs

BACKUP · V2 INDEX (TREE + INDEX), WINDOWS LAPTOP, 08.10.

V2 log rows, all nine runs

Task	Run	Steps (budget)	Verifier	Failure class	Note
ContactsAddContact	1/3	10 (12)	FAIL	lost_value	"Hugo" typed, "Pereira" never; saved; reported complete
ContactsAddContact	2/3	10 (12)	FAIL	lost_value	"Mariam" typed, "Chen" never; saved; reported complete
ContactsAddContact	3/3	10 (12)	FAIL	lost_value	"Isla" typed, "Martin" never; saved; reported complete
MarkorCreateNote	1/3	10 (16)	PASS		
MarkorCreateNote	2/3	14 (16)	PASS		3 taps on the extension field, then the type spinner: "Plain Text"
MarkorCreateNote	3/3	11 (16)	PASS		
MarkorCreateNoteAndSms	1/3	18 (18)	FAIL	grounding (?)	partial credit 0.5; 4 actions without effect; step limit
MarkorCreateNoteAndSms	2/3	16 (18)	FAIL	lost_value	".txt" typed into the name field, type left at .md; SMS sent; 0.5
MarkorCreateNoteAndSms	3/3	14 (18)	FAIL	false_done	note and SMS done, reported complete, checker 0.5

Classes without (?) were reviewed against the run protocol and are marked `failure_reviewed = yes` in `runs_windows/log.csv`. The runner had suggested `wrong_app` for five of these because of one blocked home-screen tap each; that tap was a detour, not a wrong app.

V3 stacks the remaining fixes. Several changes at once, so not attributable.

- Wait until the screen is stable before observing (`too_early`)
- "Your action changed nothing" feedback and a loop guard: same action on an unchanged screen is blocked the third time
- Status bar with scratch notes written by code (`lost_value`)
- Check step before done: the model is asked a second question, with a veto budget (`false_done`)

Result: 0 / 9 on both machines. The check step accepted the missing surname 3 of 3 times; the loop guard and the "no effect" hint fired on Markor's Save, which autosaves without a visible change.

Skills, user memory, RAG: what we built and what we left out

Concept	In our agent	Why
Skills	Built, switched off in all six versions. One tip file per app, added to the prompt only while that app is open	Tips cost tokens only when relevant. A tip changes the agent, so it would break the V1 → V2 comparison
User memory	Not built. The only memory is the note line inside one run V3	Every run starts fresh with new task values. Nothing is worth keeping between runs
RAG	Not built	Nothing to retrieve: the agent needs the current screen, and the tree delivers it every step

The reward comes from the environment, never from the agent's own report

Reward question	Our answer
Source: who decides?	Verifiable rules: AndroidWorld's checker reads the device state. "The most reliable and the cheapest to run"
Information: how much?	One number, 1 or 0; the checker also gives partial credit (0.5 when the note exists but the SMS was not sent)
Timing: when?	Once, at the end of the run
Path: is every route acceptable?	An outcome reward is blind to the path, so we log what it cannot see: steps used, failure class, recording

46%

of a V2 run is waiting for the emulator, not the model (102 s per run, 55 s of it model time)

Reward hacking, our version: status: complete without checking is the boat going in circles. The agent tries it (false_done); the verifier does not pay for it.

BACKUP · WHY WE DID NOT TRAIN

Same loop as one RL training step. Our update went into the harness, not the weights.

One RL training step	Our evaluation
1 Attempts: 16 sandboxes	3 runs per task, fresh seeds
2 Reward: a script checks the final state	AndroidWorld's verifier, 1 or 0
3 Compare within the group	V1 vs. V2 on the same task instances
4 Update the weights	update the harness: one config change, re-run

What we observe	Rule of thumb	Us
Format or schema unstable?	SFT	no: 0 invalid replies in 27 Windows runs, 1 in 54 Mac runs; the schema does it
Domain knowledge missing?	Mid-training / retrieval	no: the screen is the knowledge
Non-zero success, reliable reward, good strategy rare	RL	yes, this is our case. But one attempt costs 2 to 19 minutes on a real emulator

"A cheap simulator favours classic RL; costly or irreversible interaction favours an LLM agent." And: "if your knowledge and your instructions are good enough, you do not need to do RL at all."

What code reads out of the goal, and how it decides "typed"

Rule (in order)	Example value
free text after "with the following text:"	Actions speak louder than words.
quoted strings	"Isla Martin"
file name: stem must be typed, extension shown as a hint	brave_nest_2023_07_27 + .txt
phone numbers, compared by the last 7 digits	+19371228774
runs of capitalised words, each word its own part, minus app names and verbs	Mariam, Chen

Match: case, spaces, dashes and a trailing dot ignored; all typed chunks are also joined, so a value typed in two pieces counts.

VALUES FROM THE GOAL (tracked by code from what you typed):

- file "brave_nest_2023_07_27.txt" = name "brave_nest_2023_07_27" + extension ".txt" (if there is a separate extension field, the extension goes there): typed
- "Actions speak louder than words.": NOT typed yet

- Shown every step as the last block of the prompt
- "complete" with a value missing → rejected, at most 2 times per run, no model call
- In V4 the rejection never happened; the block itself changed behaviour

Two code rules, their firing conditions, and why the failing runs never met them

Rule	Fires when (all of)	Effect	In the 9 V6 runs
clear prefilled text	input_text into an editable element · the element shows text that is not its hint · the new text does not continue it · the agent has not typed it before	select all + delete before typing (clear_text on the device action)	0 ×: the extension-field loop is 8 to 14 clicks on ".md", never an input_text
done hint	a tap on Save / Send / SMS / OK / Done / Create / Add · every goal value typed (B6) · no action error · the screen changed	history line gets: "if the goal is fully done, reply status complete now instead of pressing it again"	1 × (Note + SMS run 3, after "SMS"; the agent pressed again). On Markor "Save" the screen never changes, so 0 ×

Both rules were written from the V4/V5 logs and gated on a condition the failing trajectories do not meet. The "screen changed" gate is V3's autosave mistake, repeated. Fix: fire on the commit tap itself.

How the suggested failure class is computed: root cause first, then symptom

Order	Class	Signal in the trajectory (counted by code, no model)
1	wrong_app	open_app for an app off the list, an action outside the allowed apps, or home-screen taps and the agent never entered an app
2	lost_value	typed text that is not in the goal, or some goal values typed and others never (B6) while the agent said "complete"
3	grounding	≥ 2 actions without screen change, or a loop-guard block, or a tap on no element plus one no-effect action
4	too_early	an effect that showed only after the extra wait (measured only with wait_for_stable, i.e. V3)
5	false_done	"complete" reported, verifier says FAIL, none of the above
else	grounding (?)	out of steps with no clearer signal: evidence says "check the recording"

Every row in `log.csv` carries the evidence string ("goal value(s) never typed: 'Martin'; 2 action(s) did not change the screen") and a `failure_reviewed` flag: the suggestion can be checked, not just believed.

Payment ban: two layers, six deny rules, no off switch

Layer 1 · app scope

- open_app only by exact name from the allow-list (Contacts, Markor, Simple SMS Messenger, ...)
- Any action while a package outside the scope is in front is blocked, except leaving it
- Home-screen taps are blocked: apps are opened by name, never by searching the launcher

Layer 2 · deny rules, checked before every action

- a opening an app that exists to buy or pay
- b anything but leaving while such an app is visible
- c anything but leaving on a payment form (a payment field on screen, or two payment signals)
- d taps and typing on, or next to, an element labelled pay / buy / checkout / add to cart: every element under the tap point is hit-tested, and the row around the label too
- e typing or Enter without a known target while a payment element is on screen
- f typing a card number (13 to 19 digits, Luhn check) or an IBAN (mod-97 check), in one go or in consecutive chunks

A blocked action costs a step and tells the model why: "BLOCKED by the payment guardrail: ... Go back and continue without paying."

One config file per version: what each one switches on

	V1	V2	V3	V4	V5	V6
model	4B	4B	4B	4B	8B	8B
observation / grounding	screenshot / coordinate	tree / index	tree / index	tree / index	tree / index	tree / index
wait_for_stable	.	.	on	.	.	.
screen_change_hint	.	.	on	.	.	.
loop_guard (abort after 3)	.	.	on	.	.	.
status_bar	.	.	on	.	.	.
verify_before_done	.	.	on	.	.	.
goal_tracker	.	.	.	on	.	.
replace_prefilled_text, done_hint	.	.	.	.	.	on

Fixed in every version

temperature / seed	0 / 42
num_ctx / max output	8,192 / 400 tokens
request timeout	300 s → ERROR
parse retries	2, with the error in the prompt
history lines kept	8
tree elements max	70, status bar dropped
screenshot max side	1,024 px
step budget	min(20, complexity × 10)
done rejections max	2 (V3 check step, V4 veto)
skills, payment ban	off / always on

BACKUP · TECHNICAL · MACBOOK AIR M4, ALL 54 SCORED RUNS

Success and cost, V1 to V6, same 9 task instances

	V1	V1	V2	V2	V3	V3	V4	V4	V5	V5	V6	V6
ContactsAddContact	2 / 3		1 / 3		0 / 3		0 / 3		2 / 3		0 / 3	
MarkorCreateNote	2 / 3		1 / 3		0 / 3		1 / 3		1 / 3		1 / 3	
MarkorCreateNoteAndSms	1 / 3		0 / 3		0 / 3		0 / 3		0 / 3		0 / 3	
all tasks	5 / 9		2 / 9		0 / 9		1 / 9		3 / 9		1 / 9	
model time per step	23.5 s		6.3 s		12.7 s		9.3 s		7.3 s		14.6 s*	
time per run	368 s		137 s		251 s		195 s		153 s		301 s*	
prompt tokens per call	1,906		1,008		1,200		1,051		1,009		1,043	
steps per run	12.8		12.9		14.6		14.8		13.6		14.4	
invalid model replies	0		0		0		0		1		0	

* V6 timed while the laptop was under other load (first steps up to 76 s, later runs 10 to 15 s); same model and prompt size as V5. 11 ERROR runs (all V3, emulator crashes) re-run with the same seed and excluded from every rate. Ollama 0.35.1; model digests are recorded per run in meta.json.

BACKUP · TECHNICAL · SAME SEED, SIX VERSIONS

The same nine task instances through every version (steps used)

task	seed	V1	V2	V3	V4	V5	V6
Contacts · Hugo Pereira	1155463587	PASS 11	PASS 11	FAIL 12	FAIL 12	FAIL 12	FAIL 12
Contacts · Mariam Chen	1324763856	FAIL 12	FAIL 12	FAIL 11	FAIL 12	PASS 12	FAIL 12
Contacts · Isla Martin	596600690	PASS 11	FAIL 9	FAIL 12	FAIL 12	PASS 8	FAIL 12
Markor · wise_yacht.md	1448135622	PASS 9	FAIL 16	FAIL 16	PASS 11	PASS 11	FAIL 13
Markor · brave_nest.txt	2337133809	PASS 11	FAIL 9	FAIL 16	FAIL 16	FAIL 9	FAIL 14
Markor · quick_eagle.md	1196614525	FAIL 9	PASS 9	FAIL 10	FAIL 16	FAIL 16	PASS 14
Note + SMS · fair_ant.txt	3221046986	FAIL 18	FAIL 16	FAIL 18	FAIL 18	FAIL 18	FAIL 18
Note + SMS · fierce_violin.txt	3411394365	PASS 16	FAIL 18	FAIL 18	FAIL 18	FAIL 18	FAIL 18
Note + SMS · lively_goat.md	3564688147	FAIL 18	FAIL 16	FAIL 18	FAIL 18	FAIL 18	FAIL 17

Same seed = same name, number, file name and text. No instance passes in every version; the .txt tasks (extension field) never pass after V1. Reading across a row shows the noise; reading down a column shows the pattern.

81 scored runs: all six versions on the Mac, V1 to V3 again on Windows

Machine	Versions	Calculation	Runs
MacBook Air M4	V1 to V6	3 tasks × 3 runs × 6 versions	54
Windows laptop (CPU)	V1 to V3	3 tasks × 3 runs × 3 versions	27
Total			81

Not counted: 11 ERROR runs (emulator crashes, all Mac V3), re-run with the same seed; the setup runs in runs_setup. Mac and Windows are kept apart, not merged: different Ollama builds mean different model digests, so strictly a different model. V4 and V5 change one thing each; V6 adds two targeted fixes.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 1 OF 14

Three versions change exactly one thing; V3 and V6 bundle several

● on ○ new compared with base (ring)

	V1 starting point	V2 base V1 1 change	V3 base V2 5 changes	V4 base V2 1 change	V5 base V2 1 change	V6 base V5 2 changes
Tree + index instead of screenshot + pi...	●	○	●	●	●	●
8B instead of 4B model	●	●	●	●	○	●
Wait for a stable screen	●	●	○	●	●	●
"Action had no effect" hint	●	●	○	●	●	●
Loop guard	●	●	○	●	●	●
Status bar + scratch notes	●	●	○	●	●	●
Check step before "complete"	●	●	○	●	●	●
Goal-value tracker + veto	●	●	●	○	●	●
Clear prefilled field	●	●	●	●	●	○
"Complete" hint after save	●	●	●	●	●	○
Step limit, payment ban, app list	●	●	●	●	●	●

Dot = mechanism on, ring = new compared with the base version. Only V1→V2, V4 and V5 are clean single-factor tests.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 2 OF 14

Note + SMS is passed exactly once across all versions and both machines

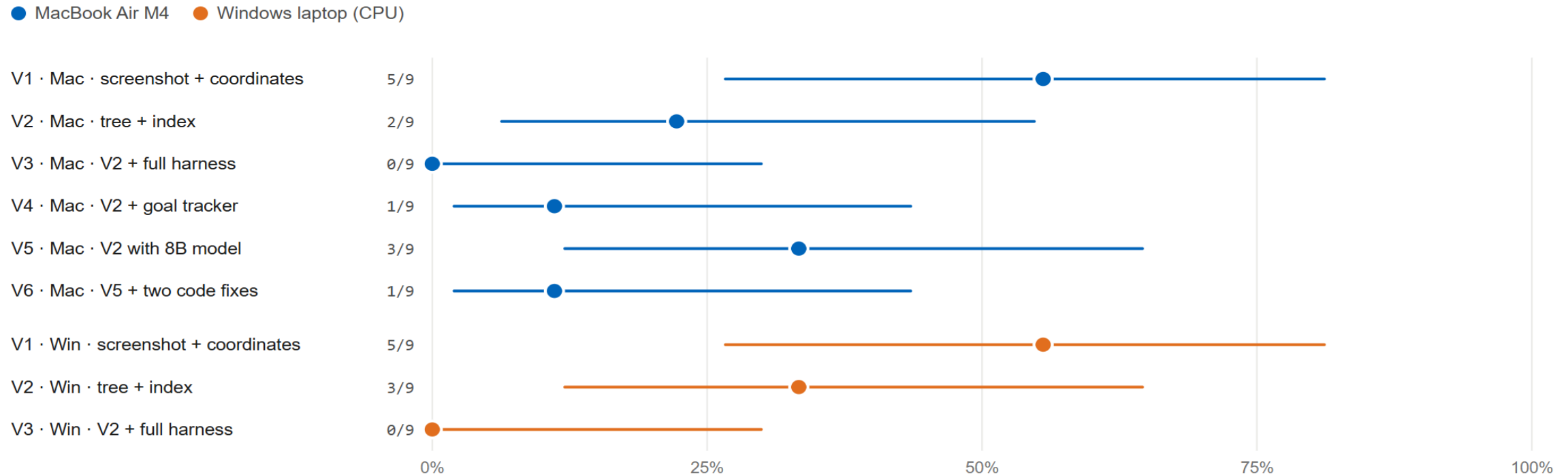
■ PASS ■ Goal reached, never “complete” ■ “Complete”, goal missed ■ Neither

	MacBook						Windows		
	V1	V2	V3	V4	V5	V6	V1	V2	V3
Add contact Seed 1155...	PASS	PASS	“Complete”, goal missed	Neither	Neither	Neither	PASS	“Complete”, goal missed	Neither
Seed 1324...	Goal reached, never “complete”	Neither	“Complete”, goal missed	Neither	PASS	Neither	PASS	“Complete”, goal missed	“Complete”, goal missed
Seed 5966...	PASS	“Complete”, goal missed	“Complete”, goal missed	Neither	PASS	Goal reached, never “complete”	PASS	“Complete”, goal missed	“Complete”, goal missed
Markor note Seed 1448...	PASS	Goal reached, never “complete”	Goal reached, never “complete”	PASS	PASS	“Complete”, goal missed	PASS	PASS	Goal reached, never “complete”
Seed 2337...	PASS	“Complete”, goal missed	Neither	Neither	“Complete”, goal missed	“Complete”, goal missed	PASS	PASS	Neither
Seed 1196...	“Complete”, goal missed	PASS	Neither	Neither	Goal reached, never “complete”	PASS	“Complete”, goal missed	PASS	Neither
Note + SMS Seed 3221...	Neither	“Complete”, goal missed	Neither	Neither	Neither	Neither	“Complete”, goal missed	“Complete”, goal missed	Neither
Seed 3411...	PASS	Goal reached, never “complete”	Neither	Neither	Neither	Neither	“Complete”, goal missed	“Complete”, goal missed	Neither
Seed 3564...	Neither	“Complete”, goal missed	Neither	Goal reached, never “complete”	Neither	“Complete”, goal missed	Neither	“Complete”, goal missed	Neither

All 81 runs: rows = task instances (same seed), columns = version and machine. Four outcome types instead of PASS/FAIL.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 3 OF 14

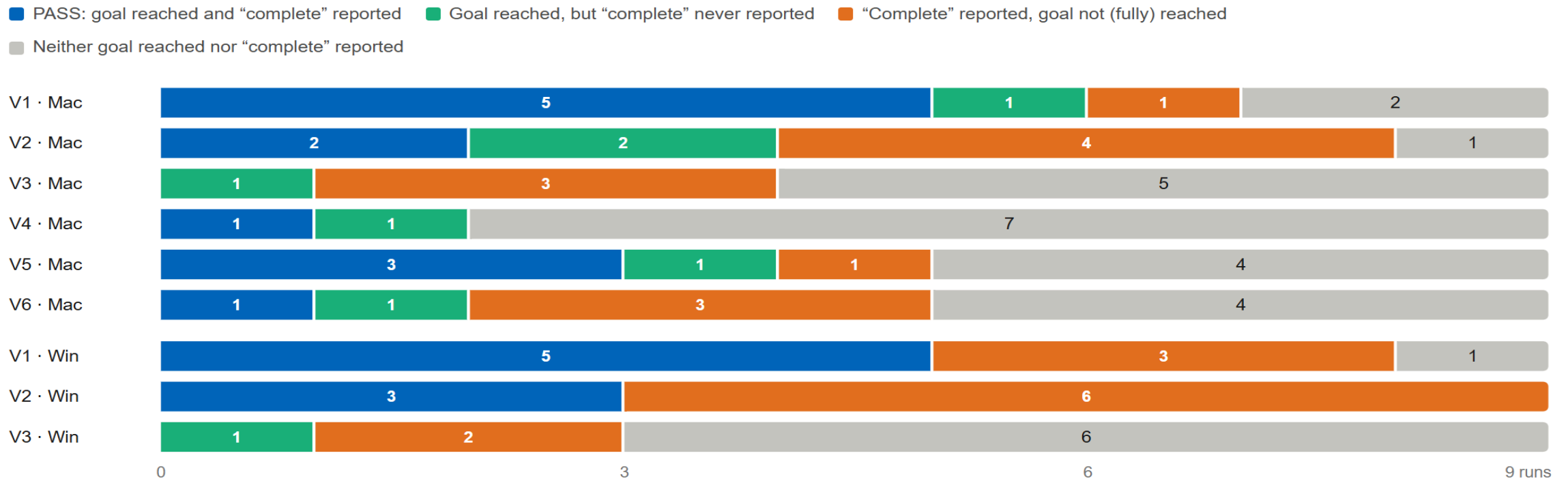
With 9 runs per version all uncertainty ranges overlap; even 5/9 and 0/9 still touch



Dot = success rate, line = 95% Wilson interval. One run moves the rate by 11 points.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 4 OF 14

Half of all failures are a wrong judgement about “complete”, not an operating error



23 of 61 failures reported “complete” with the goal missed; 8 reached the goal and never said so. Read from checker score and the “complete” flag, no manual review.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 5 OF 14

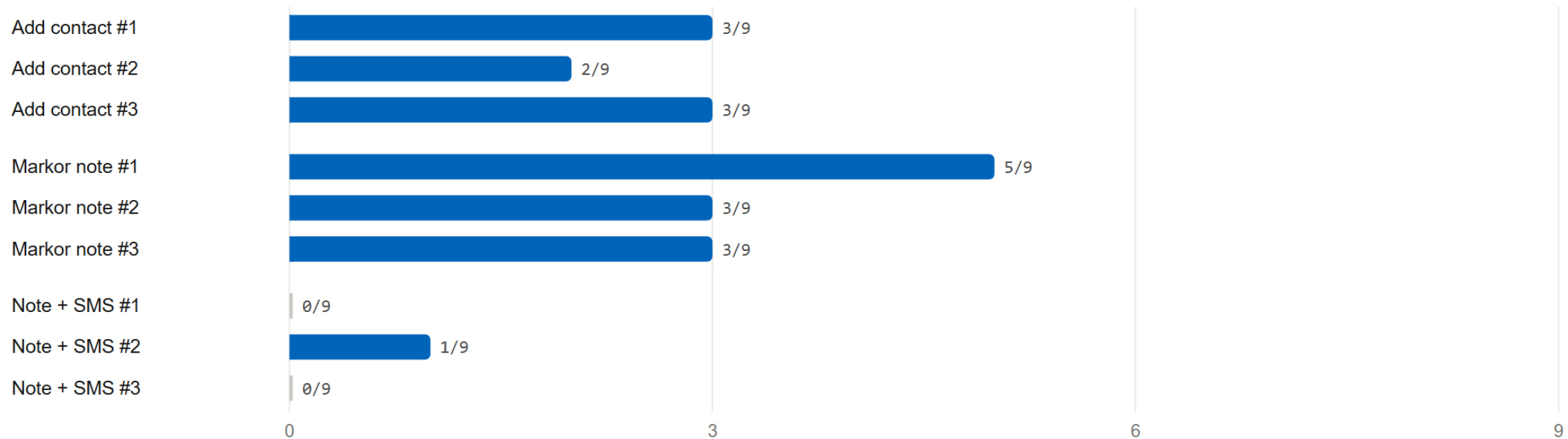
“grounding” is a catch-all: 45 of 61 failures carry it, including every run that reached the goal

	Goal reached, never “complete”	“Complete”, goal missed	Neither	Total
grounding	8	7	30	45
lost_value	0	10	0	10
false_done	0	5	0	5
too_early	0	1	0	1
wrong_app	0	0	0	0

Rows: failure class in the log; columns: what objectively happened. Only 14 of 61 classes were reviewed by hand.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 6 OF 14

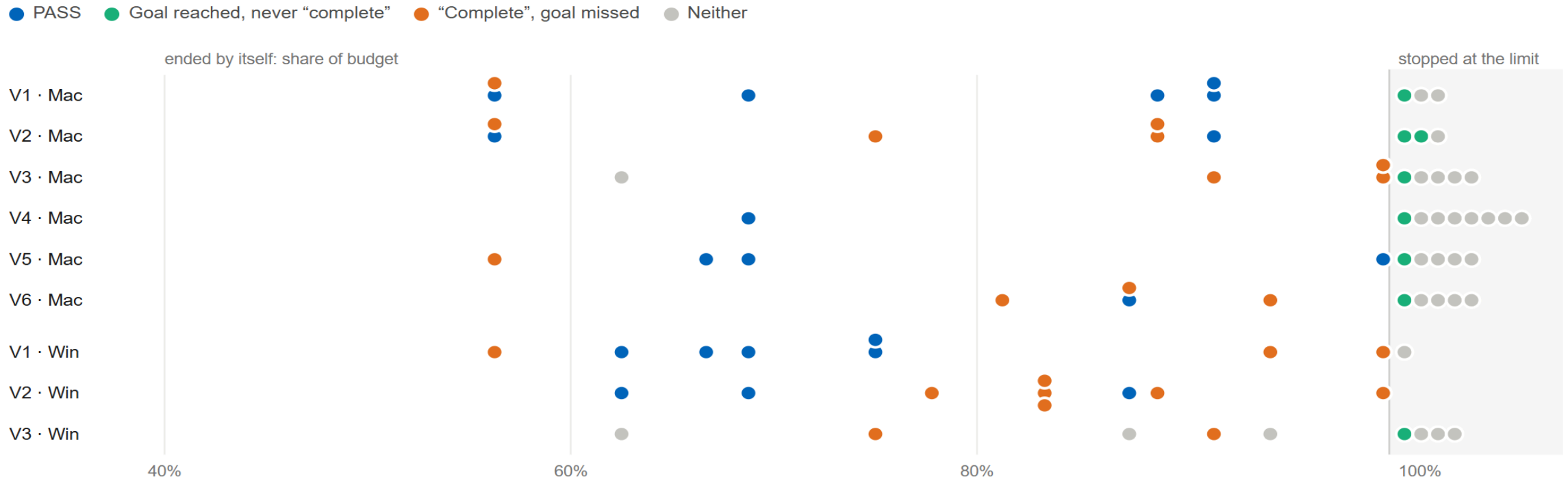
The task matters more than the version: the same instance is passed by 0 to 5 of the 9 variants



Number of version-machine combinations (out of 9) that pass each task instance.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 7 OF 14

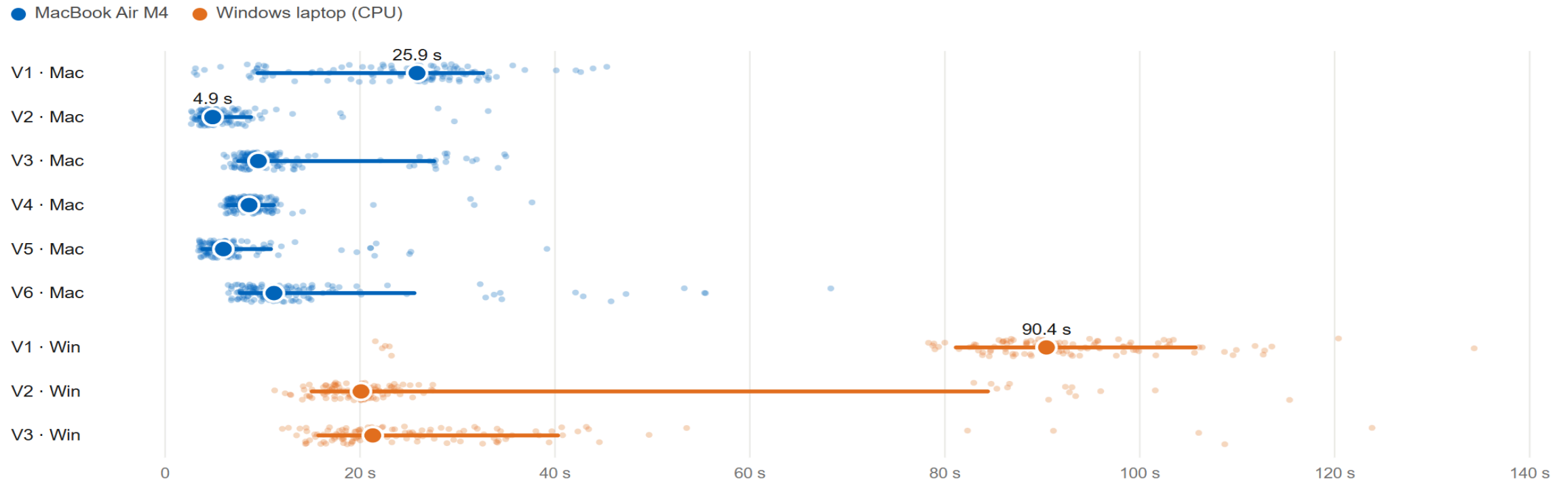
When the agent stops by itself, it is right in only 20 of 43 cases; the other 38 runs are stopped by the harness



Left: runs ended by the agent (or loop guard), by share of budget used. Right: runs stopped by the step limit in code. None of them passed.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 8 OF 14

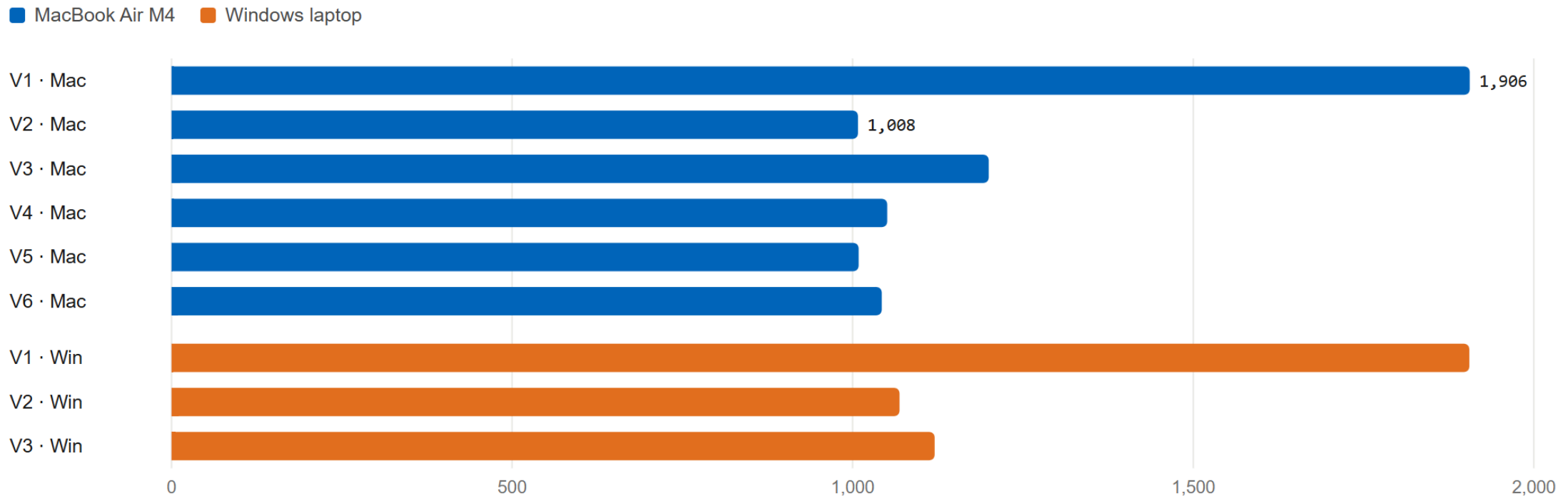
Index instead of screenshot makes each model call on the Mac about five times faster (median)



Model time per step. Small dots = single steps, large dot = median, bar = middle 80%. One shared seconds axis.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 9 OF 14

The accessibility tree halves the prompt: about 1,000 instead of 1,900 tokens per model call



Average prompt tokens per model call. Almost identical on both machines, so the time gap in B21 is hardware, not prompt.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 10 OF 14

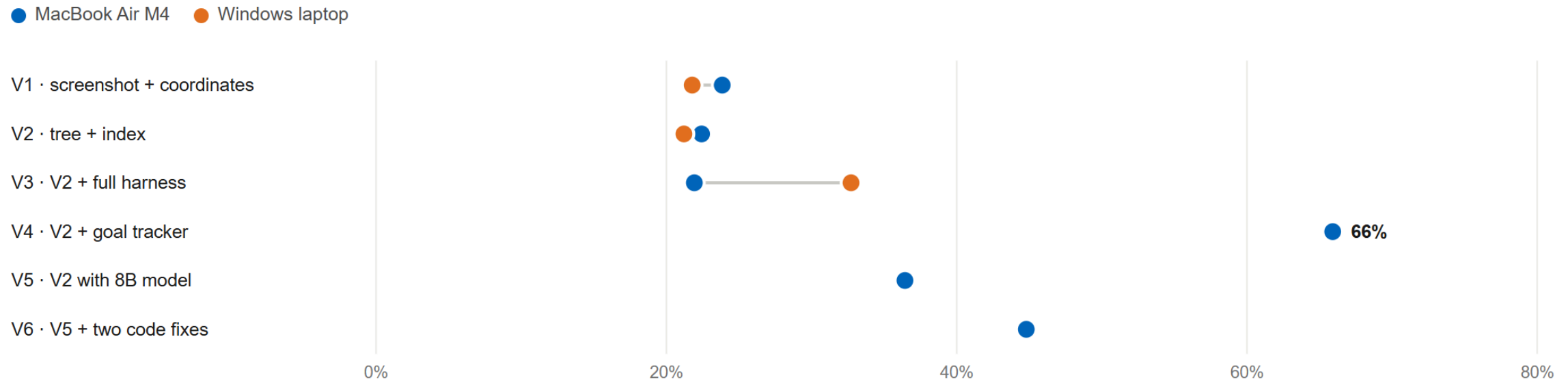
No version is both cheap and good; V1 buys its successes with the longest run time



x = average run time, y = success rate. Hollow: Mac V6, time distorted by other load.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 11 OF 14

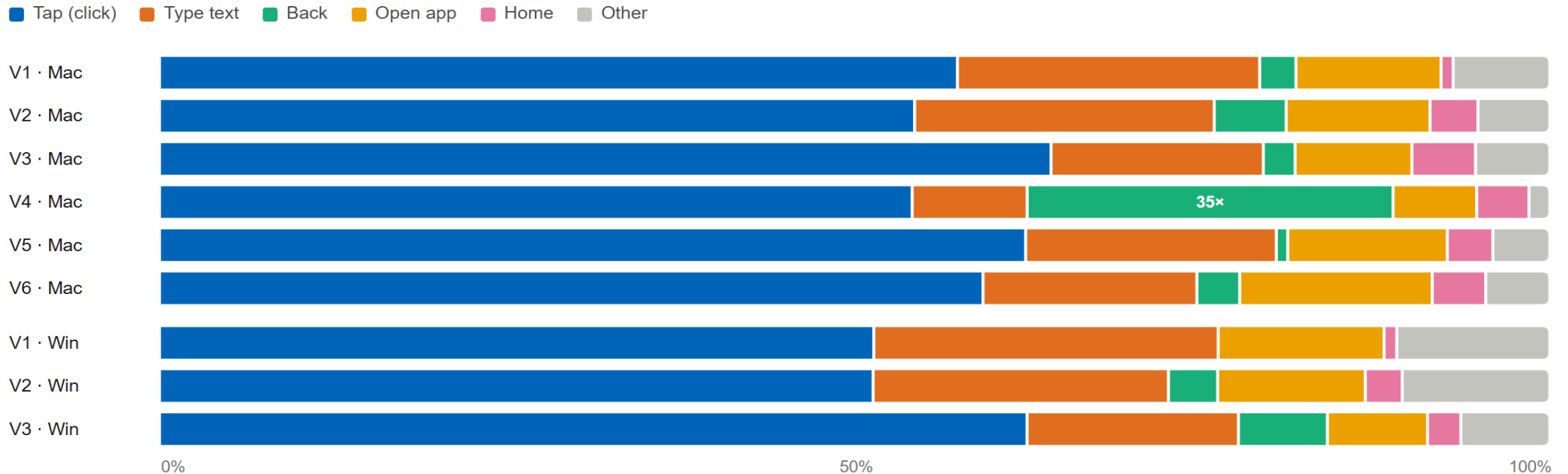
V4 spends two thirds of its steps on actions that do not change the screen



Share of executed steps without a visible screen change (fingerprint before = after), Mac and Windows.

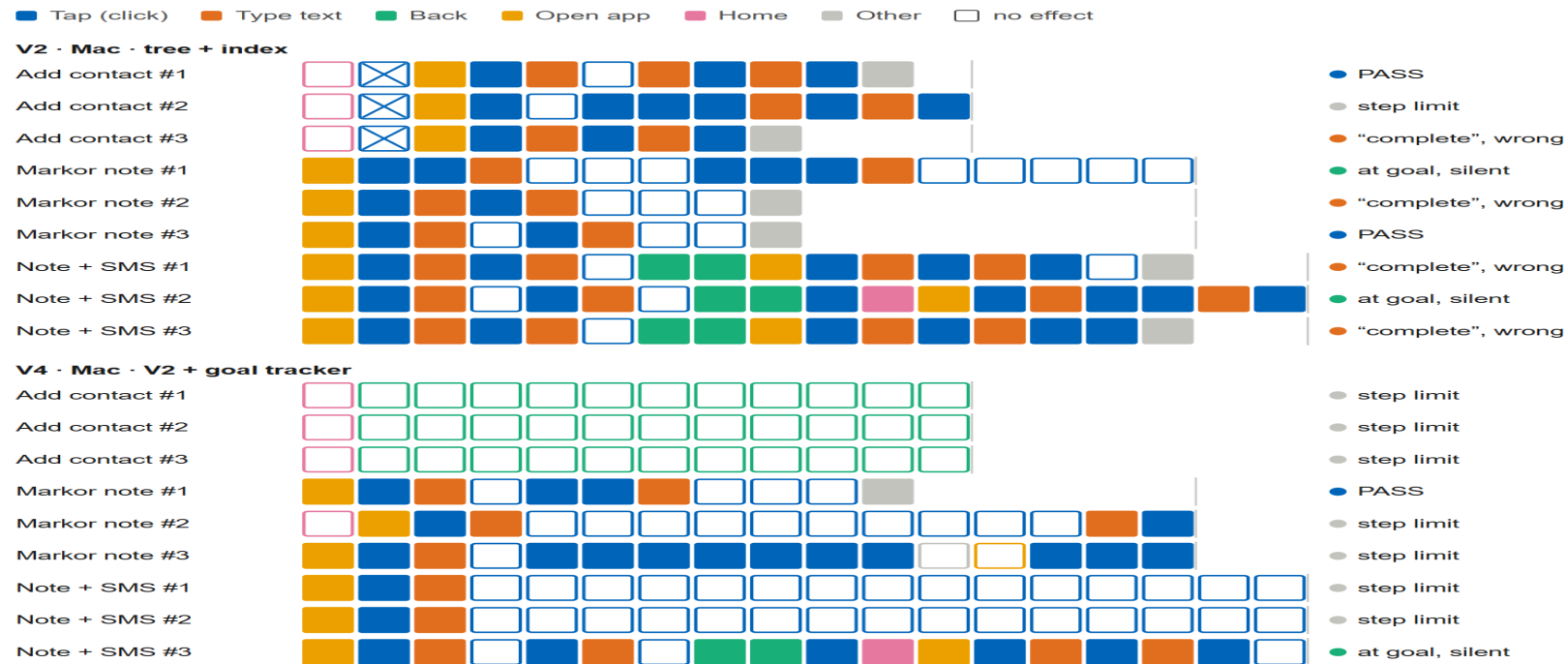
BACKUP · ALL 81 RUNS, RE-READ · FIGURE 12 OF 14

Every extra rule shifts what the agent does: V4 presses “Back” 35 times, V2 only six



All actions split by type, per version and machine. “Other” = wait, scroll, report complete.

V2 against V4 on the Mac, step by step



Each row is a run, each box a step. Hollow = screen did not change, cross = blocked by the harness. Right: outcome.

BACKUP · ALL 81 RUNS, RE-READ · FIGURE 14 OF 14

22 of 27 runs end the same on both machines, mostly because both fail

V1 · screenshot + coordinates

7 of 9 agree

	Win ↓	Mac →	
PASS		4	1
FAIL		1	3
		PASS	FAIL

V2 · tree + index

6 of 9 agree

	Win ↓	Mac →	
PASS		1	2
FAIL		1	5
		PASS	FAIL

V3 · V2 + full harness

9 of 9 agree

	Win ↓	Mac →	
PASS		0	0
FAIL		0	9
		PASS	FAIL

Per version: Windows result (rows) against Mac result (columns), same seed. Of the 10 pairs with at least one PASS, every second one flips.